

IS A RELATIONSHIP

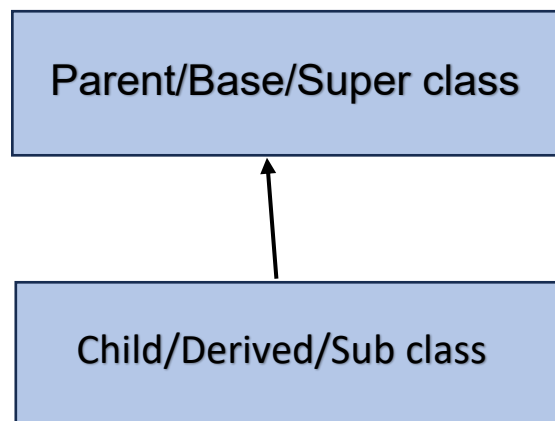
Connection between two objects is called as relationship in Java. Relationship is of two types.

Has A Relationship: Connection between two objects exists as dependency between them. The two classes are called Dependent class and Another class.

Is A Relationship: Connection between two objects exists as Parent – Child relationship between classes. The two classes are called Parent/Super/Base class and Child/Sub/Derived class.

IS A RELATIONSHIP:

The relationship between two objects exists as Parent-Child relationship. The two classes are called as Parent and Child classes.



In the above pictorial representation, both classes are represented along with a solid arrow pointing to the parent class. This arrow will always point to the parent class. It is the representation in UML diagram. The same Is A relationship or Inheritance can be represented in the program using keywords.

TWO KEYWORDS:

1. **extends**: when both parent and child are same components of Java, like both parent and child are classes or both are interfaces.
2. **implements**: when parent is interface and child is a class. Interface cannot be a child in Java.

TWO TECHNIQUES:

1. **Specialization**: when object is created to a class and reference is stored in the same type. Usually, we refer this for creating an object to child class and storing the reference in child type in Inheritance.
2. **Generalization**: when object is created to a class and reference is stored in different type. Usually, we refer this as creating an object to child class and storing the reference in Parent type in Inheritance.

Definition of Inheritance:

The process of inheriting or acquiring the properties of a class from another class is called inheritance.

Characteristics of Inheritance:

1. The parent class has its own properties.
2. The child class has its own properties.
3. The properties of the parent class get inherited to the child class.
4. **private members, constructors and initializer blocks** of parent class do not get inherited to the child class.
5. The child type object reference can access the members of child class and parent class.
6. The parent type object reference can access only the parent class members but cannot access the child class members.

```
1 package sampledemo;
2
3 class Parent{
4     private int a;
5     int b;
6     String x;
7 }
8 class Child extends Parent{
9     int k;
10    String d;
11 }
12 class DemoInheritance {
13     public static void main(String[] args) {
14         Child c = new Child();
15         System.out.println(c.k);
16         System.out.println(c.x);
17         //System.out.println(c.a); // ERROR: NOT VISIBLE
18         // a is private, not inherited to child class
19         Parent p = new Parent();
20         System.out.println(p.b);
21         //System.out.println(p.d); // ERROR: d cannot be resolved or d is not a field
22         //parent type reference cannot access child class member
23     }
24 }
25
```

<terminated
0
null
0

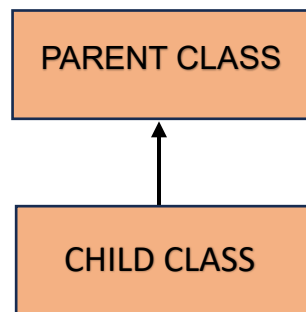
TYPES OF INHERITANCE:

There are five types of Inheritance in Java as follows:

1. SINGLE LEVEL INHERITANCE
2. MULTI LEVEL INHERITANCE
3. HIERARCHICAL INHERITANCE
4. MULTIPLE INHERITANCE
5. HYBRID INHERITANCE

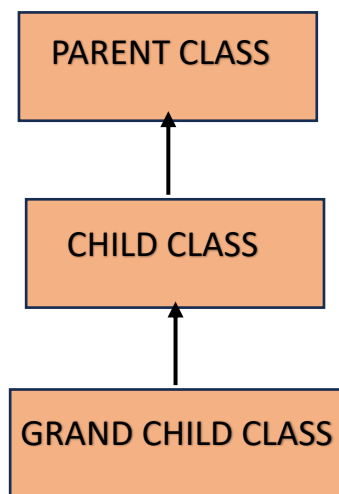
SINGLE LEVEL INHERITANCE:

Only one parent and one child in the inheritance relationship.



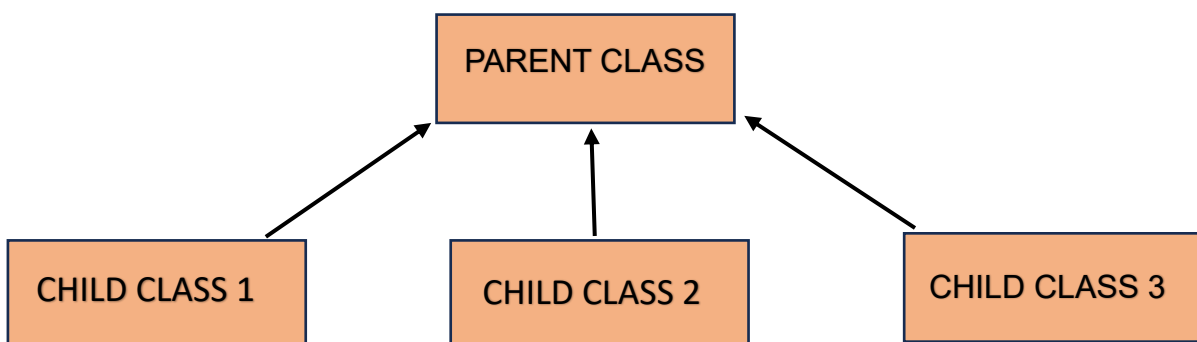
MULTI LEVEL INHERITANCE:

More than one level of parent-child relationship.



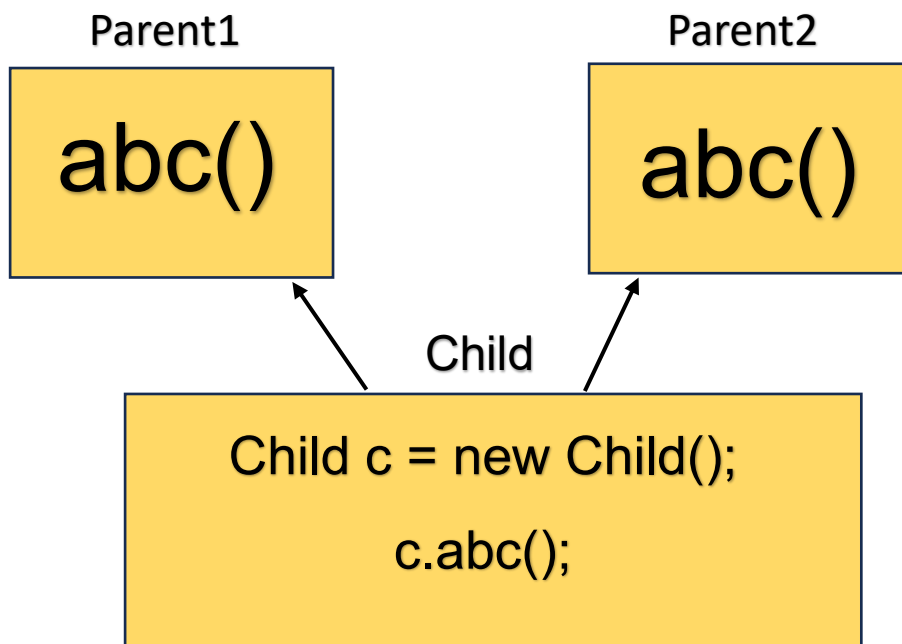
HIERARCHICAL INHERITANCE:

One parent class has more than one child classes.



MULTIPLE INHERITANCE:

If a child class has more than one parent, it is called multiple inheritance. Let's see an example for that.



In the above diagram, there are two parent classes Parent1 and Parent2. Child class inherits both the parent classes.

Scenario: Both parents have non-static method abc(), which has same signature. When we create object for the Child class and try to access the method abc() with Child type object reference, the compiler gets confused as to which method to execute. This is called as **ambiguity**.

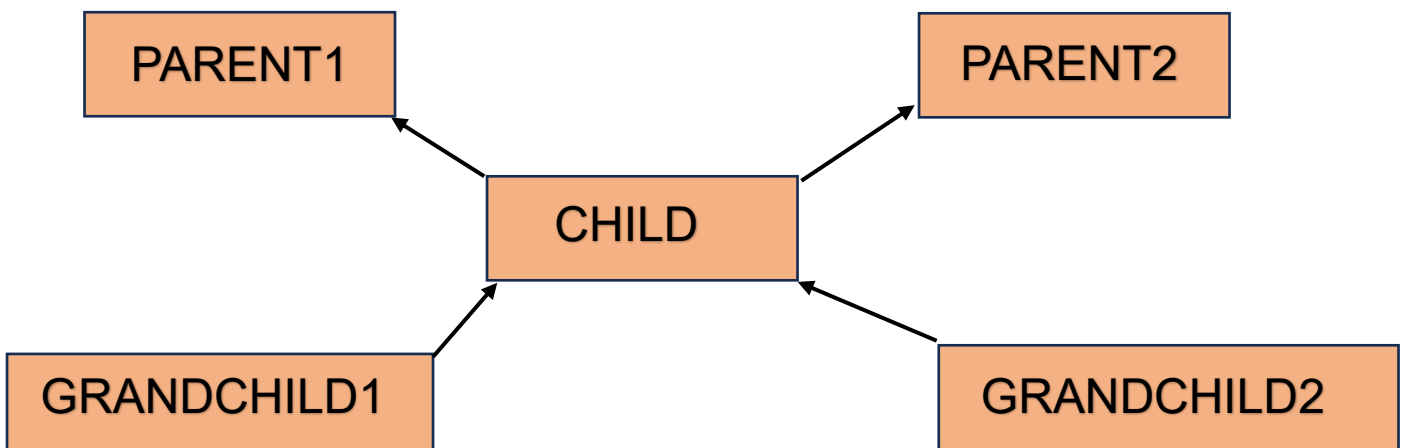
This ambiguity is referred to as **DIAMOND PROBLEM** in Java.

How to overcome Diamond problem?

By making both the parents as Interfaces, we can overcome diamond problem in Java. Multiple inheritance using classes with same method signature is not possible in Java.

HYBRID INHERITANCE:

It is a combination of both Multiple and Hierarchical inheritance.



GENERALIZATION:

The process of creating object for a class and storing the object reference in another type variable is called Generalization. This is also called as Upcasting under Non-primitive typecasting.

NON-PRIMITIVE TYPECASTING:

The process of converting one non-primitive datatype into another non-primitive datatype is called non-primitive typecasting.

It is of two types: up-casting and down-casting.

Up-casting: The process of creating an object to the child class and storing the object reference in parent type is called up-casting.

```
Parent p = new Child();
```

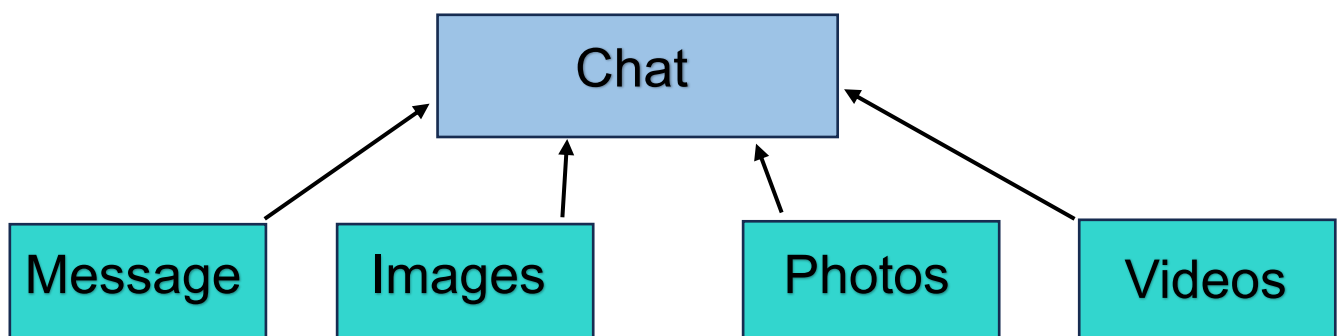
Down-casting: The process of converting the parent type object reference back to child type is called down-casting. Up-casting or generalization is mandatory to perform down-casting. Round braces () is called typecast operator.

```
Child c = (Child)p;
```

ADVANTAGE OF GENERALIZATION:

Especially during hierarchical inheritance, generalization helps in **easy handling** of various child type objects as they are stored in **same parent type object reference**.

Real-time example for up-casting and down-casting:



Generalized/Upcasted scenario:

```
Chat c1 = new Message();  
Chat c2 = new Images();  
Chat c3 = new Photos();  
Chat c4 = new Videos();
```

Down-casting for Images:

```
Images i = (Images)c2;
```

- In the above example, Chat is the parent class.
- It has many features (classes) like Message, Images, Photos and Videos.
- The objects for the child classes are created in a **generalized container** called Chat type.
- Only when changes or updates are required for any of the child class it can be down-casted and used.
- Here, we have down-casted c2 to Images type object reference 'i'.

super() – super call statement:

Constructor calling statement used to call a constructor from another class.

Characteristics of super call statement – super():

1. super() is used to call another constructor from a constructor of another class
2. super() statement can be used only inside constructor body
3. It should always be the first statement - only one can be used
4. It calls the constructor of another class
5. In all constructors of a class, super() statements can be used
6. In a constructor, both this() and super() can NOT be used

MEMBERS NOT INHERITED:

1. *Private members:*

The scope of the private members is only within the class where they are declared. Hence, they are not inherited.

Solution: By using getter and setter methods, they can be accessed in the child class.

2. *Constructors:*

The constructors are not considered as class members. Hence not inherited.

Solution: Special statement called super() – super call statement is used.

3. *Initializer blocks:*

Both static and non-static Initializer blocks do not get inherited.

Solution: They get executed implicitly.

ML static initializers: during class loading

ML non-static initializers: during object loading for every object

```

24
25 class Vegetable
26 {
27     private String type;
28     String color;
29     float price;
30     public String getType()
31     {
32         return type;
33     }
34     public void setType(String type)
35     {
36         this.type = type;
37     }
38     Vegetable(String color, float price)
39     {
40         this.color = color;
41         this.price = price;
42     }
43 }
44
45 class Potato extends Vegetable
46 {
47     int quantity;
48     float calories;
49     Potato(String color, float price, int quantity, float calories)
50     {
51         super(color, price);
52         this.quantity = quantity;
53         this.calories = calories;
54     }
55     public void potatoDetails()
56     {
57         System.out.println("*****POTATO DETAILS*****");
58         System.out.println("color of potato: "+color);
59         System.out.println("price of potato: "+price);
60         System.out.println("quantity of potato: "+quantity);
61         System.out.println("calories of potato: "+calories);
62     }
63 }
64
65 class Tomato extends Vegetable
66 {
67     Tomato(String color, float price)
68     {
69         super(color, price);
70     }
71     public void tomatoDetails()
72     {
73         System.out.println("*****TOMATO DETAILS*****");
74         System.out.println("color of tomato: "+color);
75         System.out.println("price of tomato: "+price);
76     }
77 }
78

```

```

1 package javapackage;
2
3 class GeneralizationDriver
4 {
5     public static void main(String[] args)
6     {
7         Vegetable v1 = new Potato("brown", 65.21f, 3, 246.21f); // UP CASTING
8         //v1 is object reference of parent type
9         //with parent type object reference we cannot access child class data members
10        //disadvantage of up casting
11        Potato p = (Potato)v1; // DOWN CASTING
12        //p is object reference of child type // down casting
13        p.potatoDetails();
14
15        Vegetable v2 = new Tomato("red", 24.31f); // UP CASTING
16        //v2 is object reference of parent type
17        //with parent type object reference we cannot access child class data members
18        //disadvantage of up casting
19        Tomato t = (Tomato)v2; // DOWN CASTING
20        //t is object reference of child type // down casting
21        t.tomatoDetails();
22    }
23 }

```

<terminated> GeneralizationDriver [J

```

*****POTATO DETAILS*****
color of potato: brown
price of potato: 65.21
quantity of potato: 3
calories of potato: 246.21
*****TOMATO DETAILS*****
color of tomato: red
price of tomato: 24.31

```